

Maintenance of database files for IBM i

User Guide

Contents

Introduction	3
Using the application	4
<i>Objects used in the application</i>	<i>4</i>
<i>Directories</i>	<i>4</i>
<i>Location of the application.....</i>	<i>4</i>
<i>Start of the application.....</i>	<i>5</i>
Parameters	6
<i>Application language</i>	<i>6</i>
<i>Server address</i>	<i>6</i>
<i>User name</i>	<i>6</i>
<i>Size of the window displaying the database file</i>	<i>6</i>
<i>Mark for null field values.....</i>	<i>6</i>
<i>Size of the font for printing data</i>	<i>6</i>
<i>Maximum number of records to display</i>	<i>7</i>
<i>Limit for length of displayed data fields</i>	<i>7</i>
<i>Character set - charset and Select character set for CLOB.....</i>	<i>7</i>
<i>Available character sets</i>	<i>8</i>
<i>Library with database files.....</i>	<i>9</i>
<i>Select database file</i>	<i>9</i>
<i>File member</i>	<i>9</i>
<i>Connect / check.....</i>	<i>9</i>
Run.....	10
Selection and ordering of records	11
Changing a record	13
Creating the first record in empty file	14
Changing a cell (field in a record)	15
Deletion of a record	16
Selection of columns.....	16
<i>Example</i>	<i>16</i>
Data members of physical and logical files	19
<i>Alias objects</i>	<i>19</i>
Data types CLOB and BLOB.....	20
<i>Insert a new row</i>	<i>21</i>
<i>Updating a row</i>	<i>22</i>
Filling column of CLOB type	22
<i>Overwriting text</i>	<i>24</i>
<i>Finding text by a pattern.....</i>	<i>24</i>
<i>Saving the changed text.....</i>	<i>25</i>
<i>Changing column contents.....</i>	<i>26</i>
<i>Saving column value to a file.....</i>	<i>27</i>
Filling column of BLOB type	28
<i>Changing contents of a column.....</i>	<i>30</i>
<i>Problems with big columns.....</i>	<i>32</i>

Introduction

Motivation for this application was the fact that the popular utility DFU (Data File Utility) is unable to display or print all Unicode characters, especially UTF-8, UTF-16 or UCS-2.

The application enables to display, enter and update data in database files (tables). It works with physical files (tables) and logical files (views). Only one file can be processed at a time.

Physical file should contain at least one member. Reference file without a data member displays only its header with field (column) names.

Logical file that refers to a single physical file can also be used to enter and update data.

A file (physical or logical) with multiple data members can be processed using ALIAS objects corresponding to the members.

Traditional IBM i terms are mainly used in the application. Correspondence of IBM i and SQL terms are listed in the following table.

<i>Traditional names</i>	<i>SQL names</i>
library	schema
physical file	table
field	column
record	row
unique key	primary key unique key
key	index
logical file with selection	view
DDM file for a member	ALIAS object

Programs are written in Java language and require version *Java SE 8* or higher. They cooperate with programs from package *IBM i Toolbox for Java* (or *JTOpen*). Programs were created in the operating system OS X 10.9 (Mavericks). They were subsequently tested in operating systems *OS X* up to *macOS 26* (Tahoe) and *Windows 7* up to 11 with remote internet connection to the IBM i, versions 7.1 up to 7.5.

Using the application

The application is not installed, it is delivered as a directory that can be placed at a suitable location in the computer (maybe in a flash disk). The same copy of the application works the same in systems macOS and Windows.

Objects used in the application

Objects contained in the application are placed in the current directory. This is ensured at installation of the application, that is delivered as a whole directory (folder). When the application is launched the directory becomes current.

Note: Objects contained in the application directory that are not introduced here serve for managing the project and may be removed without any harm.

Directories

- *columnfiles* - contains text files with lists of fields (columns) for the SELECT statement,
- *helpfiles* - contains this document in Czech and English languages,
- *logfiles* - contains text files *err.txt* and *out.txt*, to which redirected output of the files System.err and System.out (i.e. from the console),
- *paramfiles* - contains the file *U_Parameters.txt* with application parameters,
- *selectfiles* - contains text files with data for selection and ordering of records in database files.
- File *jt400-21.0.4.jar* - contains classes from the package IBM i Toolbox for Java.
- File ***IBMiSqlUpdate.jar*** - contains Java classes of the application and *launches the application*.

Files *err.txt* and *out.txt* in the directory *logfiles* serve to find the cause of an error in program.

The directories *selectfiles* and *columnfiles* can be deleted without damage, they are created if needed. Contents of these files should not be changed manually.

Location of the application

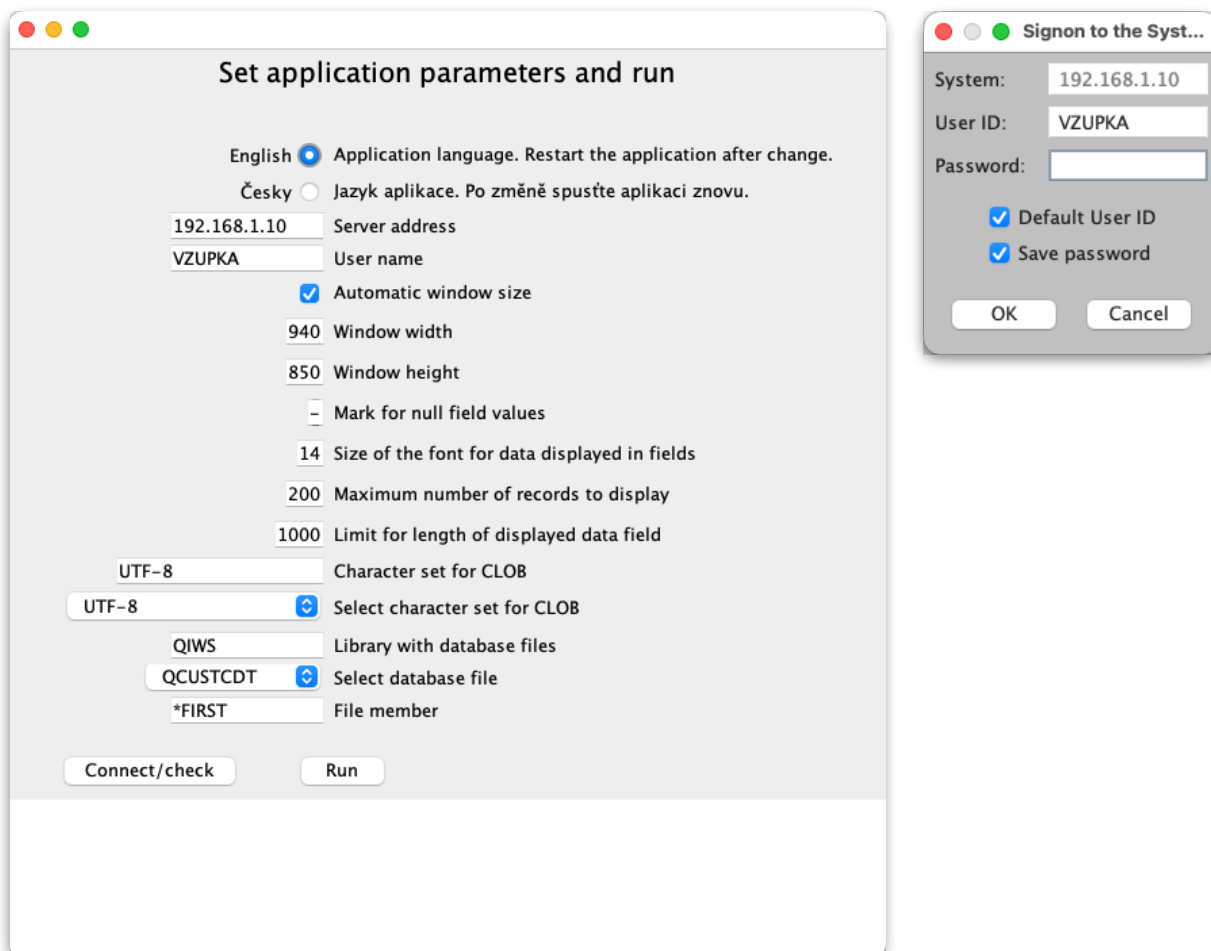
The application directory can be placed anywhere and possibly renamed. Find the file ***IBMiSqlUpdate.jar*** in the directory. Shortcuts (aliases) can be created from this file and placed somewhere.

Start of the application

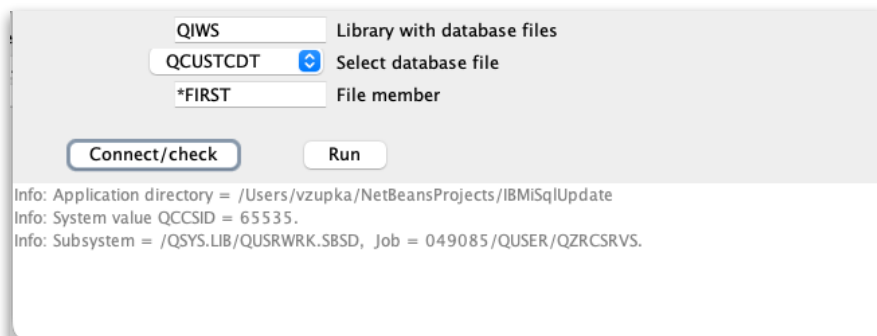
The application starts by *double click* on a shortcut (or the original) file *IBMiSqlUpdate.jar*.

The window *Set application parameters and run* is displayed. Default parameters are contained in the input fields. The user can change them. Only *Select database file* cannot be used until the application is connected to the server.

After *Server address* and *User name* is set, the user presses the button *Connect/check* and *signon dialog* for entering password to access to the IBM i system appears.



After successful signon, the user can still adjust parameter values, including *Library with database files* and *Select database file*, save them using the *Connect/check*.



Parameters

Note: If the user enters some parameters incorrectly, then after attempt of connecting to the server (*Connect/check*), messages are reported in the lower part of the window. Messages are reported just as that, when the user enters incorrect data after having already connected the server. Button *Run* is now disabled. It is enabled only after correction of data.

X1000 Limit for length of displayed data field

UTF-8 Character set for CLOB

UTF-8 Select character set for CLOB

QIWSA Library with database files

QCUSTCDT Select database file

*FIRST File member

Connect/check Run

! Application directory = /Users/vzupka/NetBeansProjects/IBMiSqlUpdate
! System value QCCSID = 65535.
! Subsystem = /QSYS.LIB/QUSRWRK.SBSD, Job = 055071/QUSER/QZRCRSVS.
! Invalid library name - QIWSA
? Correct application parameters or the library name. - For input string: "X1000"

Application language

The application can be processed in English (en_US) or Czech (cs_CZ) localization. The localization concerns titles, messages, button labels. The user can choose the localization by clicking the button. The selected option is applied only after ending the application and launching it again.

Server address

An IP address in dot form or domain form. This address is copied into the dialog window *Signon to the System*.

User name

Name of the profile that has the authority to write and change data in database files. This name is copied into the dialog window *Signon to the System*.

Size of the window displaying the database file

If the box *Automatic window size* is checked, the window is accommodated to the size of the displayed results. Otherwise the window will have dimensions specified in input fields *Window width* and *Window height*.

Mark for null field values

The user enters a text (of arbitrary length) that will be shown everywhere the field value is NULL. This symbol is also used for entering the NULL value in the table cell or input field. It is *not used* for large objects CLOB and BLOB.

Size of the font for printing data

This entry represents the number of points on the display. It determines size of letters in titles and cells in table cells and also in input data fields.

Maximum number of records to display

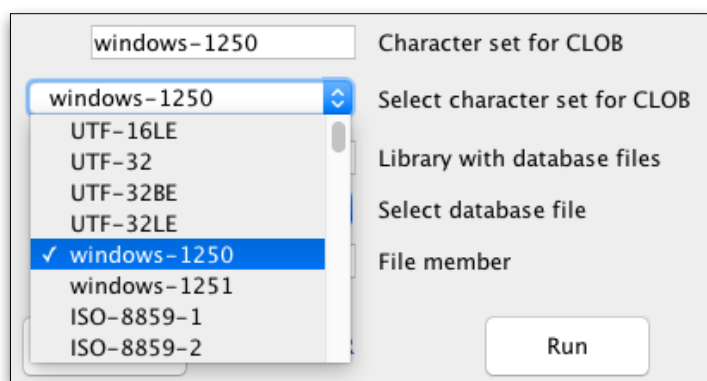
This entry represents maximum number of records that are selected from the database file and that will be shown in the window table.

Limit for length of displayed data fields

This entry represents maximum length of the table field (cell) for a data field (column) value.

Character set - charset and Select character set for CLOB

The user defines the character set for processing columns that have type of CLOB (Character Large Objects). The character set concerns the *text file* from which data is transferred to the column (input) or the text file to which data is transferred from the column (output). The user can enter the symbol of the character set to the input field manually or use the button *Select character set for CLOB*. The user selects a symbol for a character set from the menu.



Contents proper of a CLOB column is coded in java **Unicode**. (see <https://docs.oracle.com/javase/tutorial/i18n/text/unicode.html>).

Available character sets

There are many character sets supported in Java, all are listed in the menu of the button (see below). The most frequent charsets are at the beginning of the list:

UTF-8	Eight-bit Unicode (or UCS) Transformation Format
US-ASCII	American Standard Code for Information Interchange
UTF-16	Sixteen-bit Unicode (or UCS) Transformation Format, byte order identified by an optional byte-order mark
UTF-16BE	Sixteen-bit Unicode (or UCS) Transformation Format, big-endian byte order
UTF-16LE	32-bit Unicode (or UCS) Transformation Format, little-endian byte order
UTF-32	32-bit Unicode (or UCS) Transformation Format, byte order identified by an optional byte-order mark
UTF-32BE	32-bit Unicode (or UCS) Transformation Format, big-endian byte order
UTF-32LE	32-bit Unicode (or UCS) Transformation Format, little-endian byte order
windows-1250	Windows Eastern European (Latin 2)
windows-1252	Windows Latin-1
ISO-8859-1	ISO-8859-1, Latin Alphabet No. 1
ISO-8859-2	Latin Alphabet No. 2

Note: List of the charsets supported in Java can be found in the documentation on the address <http://docs.oracle.com/javase/8/docs/technotes/guides/intl/encoding.doc.html>. List of all charsets is in the publications *IANA Charset Registry* on the address <http://www.iana.org/assignments/character-sets/character-sets.xhtml>.

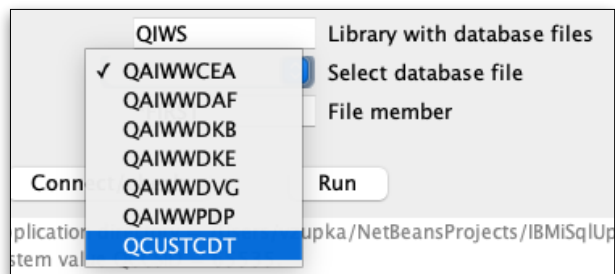
The *contents of the CLOB column* is stored in the database in character set corresponding to the column type specified when the database file was created.

Library with database files

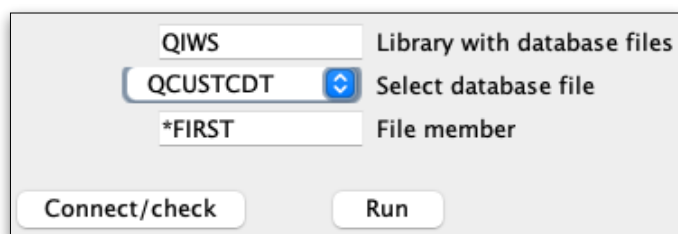
The name convention “*system*” is used to process SQL statements. Enter the name of the library containing database files to work with.

Select database file

The user presses the button and selects a name from the name list, say QCUSTCDT.



The selected name is then shown as the name of the button.



File member

The input field *File member* is constantly set to the value *FIRST. The application works with files of **PF-DATA** type, that is without source files (PF-SRD type). The user may overwrite the value *FIRST by name of a data member of the file if it exists.

See the chapter [Data members of physical and logical files](#) below.

Connect / check

Button Tlačítko *Connect/check* connects the application to the server, checks parameters and reports eventual error messages.

Run

If there are no errors in parameters, the button *Run* shows a window with a table containing maximum number of the first records selected from the file (see *Parameters* above). File QCUSTCDT in library QIWS was chosen in this example because it is contained in every IBM i server.

Table QIWS/QCUSTCDT

Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).

RRN	CUSNUM	LSTNAM	INIT	STREET	CITY	STATE	ZIPCOD	CDTLMT	CHGCOD	BALDUE	CDTDUE
1	938472	Henning	G K	4859 Elm Ave	Dallas	TX	75217	5000	3	37.00	0.00
2	839283	Jones	B D	21B NW 135 St	Clay	NY	13041	400	1	100.00	0.00
3	392859	Vine	S S	P0 Box 79	Broton	VT	5046	700	1	439.00	0.00
4	938485	Johnson	J A	3 Alpine Way	Helen	GA	30545	9999	2	3987.50	0.50
5	397267	Tyron	W E	13 Myrtle Dr	Hector	NY	14841	1000	1	0.00	0.00
6	389572	Stevens	K L	208 Snow Pass	Denver	CO	80226	400	1	58.75	0.50
7	846283	Alison	J S	787 Lake Dr	Isle	MN	56342	5000	3	10.00	0.00
8	475938	Doe	J W	59 Archer Rd	Sutter	CA	95685	700	2	250.00	1.00
9	693829	Thomas	A N	3 Dove Circle	Casper	WY	82609	9999	2	0.00	0.00
10	593029	Williams	E D	485 SE 2 Ave	Dallas	TX	75218	200	1	25.00	0.00
11	192837	Lee	F L	5963 Oak St	Hector	NY	14841	700	2	489.50	0.50
12	583991	Abraham	M T	392 Mill St	Isle	MN	56342	9999	3	500.00	0.00

Enter condition WHERE for row selection and press Refresh.

Enter condition ORDER BY for row ordering and press Refresh.

SELECT RRN(QCUSTCDT) as RRN, CUSNUM, LSTNAM, INIT, STREET, CITY, STATE, ZIPCOD, CDTLMT, CHGCOD, BALDUE, CDTDUE
FROM QIWS/QCUSTCDT
FETCH FIRST 200 ROWS ONLY

ExitInsert new rowEdit selectedRefreshDelete selectedSelect columns

Complete SELECT statement is shown under two text frames (above the button row). The list of columns and maximum number of records is apparent from the statement. The expression *RRN(QCUSTCDT) as RRN* represents relative number of the record in the file. It is displayed for information in the first column of the table in blue color. It is used for updating or deleting of the record and its value cannot be changed.

Selection and ordering of records

The first display of the file is a result of the SELECT statement without using clauses WHERE an ORDER BY, so the first unordered records are selected in the maximum number specified in Parameters. More accurate, the records are ordered by the relative record number named as RRN. This order corresponds to the arrival sequence in which the records were written to the file.

To select and order records input frames under the displayed table can be used.

- To select records, clause **WHERE** of the SELECT statement is entered in the first frame.
- To order records, clause **ORDER BY** of the SELECT statement is entered in the second frame.

Refresh button performs new selection and/or ordering of records.

For example, if the entry in the first field is **BALDUE >= 30** and **CHGCOD < 3** and the entry in the second field is **BALDUE DESC**, the following window is displayed with selected and ordered records and the corresponding SELECT statement.

Table QIWS/QCUSTCDT

Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).

RRN	CUSNUM	LSTNAM	INIT	STREET	CITY	STATE	ZIPCOD	CDTLMT	CHGCOD	BALDUE	CDTDUE
4	938485	Johnson	J A	3 Alpine Way	Helen	GA	30545	9999	2	3987.50	0.50
11	192837	Lee	F L	5963 Oak St	Hector	NY	14841	700	2	489.50	0.50
3	392859	Vine	S S	P0 Box 79	Broton	VT	5046	700	1	439.00	0.00
8	475938	Doe	J W	59 Archer Rd	Sutter	CA	95685	700	2	250.00	1.00
2	839283	Jones	B D	21B NW 135 St	Clay	NY	13041	400	1	100.00	0.00
6	389572	Stevens	K L	208 Snow Pass	Denver	CO	80226	400	1	58.75	0.50

Enter condition WHERE for row selection and press Refresh.
BALDUE >= 30 and CHGCOD < 3

Enter condition ORDER BY for row ordering and press Refresh.
BALDUE DESC

SELECT RRN(QCUSTCDT) as RRN, CUSNUM, LSTNAM, INIT, STREET, CITY, STATE, ZIPCOD, CDTLMT, CHGCOD, BALDUE, CDTDUE
FROM QIWS/QCUSTCDT
WHERE BALDUE >= 30 and CHGCOD < 3
ORDER BY BALDUE DESC
FETCH FIRST 200 ROWS ONLY

ExitInsert new rowEdit selectedRefreshDelete selectedSelect columns

The expressions in frames are stored in a text file with ending **.sel** in the directory *selectfiles*.

Contents of the text file has the following form.

```
selection;  
ordering
```

where *selection* is either empty text or the expression for WHERE clause and *ordering* is either empty text or the expression for ORDER BY clause. The two parts are separated by semicolon.

The file from the example is named **QIWS-QCUSTCDT.sel** and contains

```
BALDUE >= 30 and CHGCOD < 3;  
BALDUE desc
```

Note 1: As an entry in both fields, the expression **RRN(QCUSTCDT)** denoting the relative record number can be used.

Note 2: To select a binary field (type BINARY, VARBINARY), function **HEX()** must be used because the value must be written as couples of hexadecimal characters. For example, if the field BIN01 of the BINARY type the expression for WHERE is entered as follows.

```
hex(BIN01) like '%cd%'
```

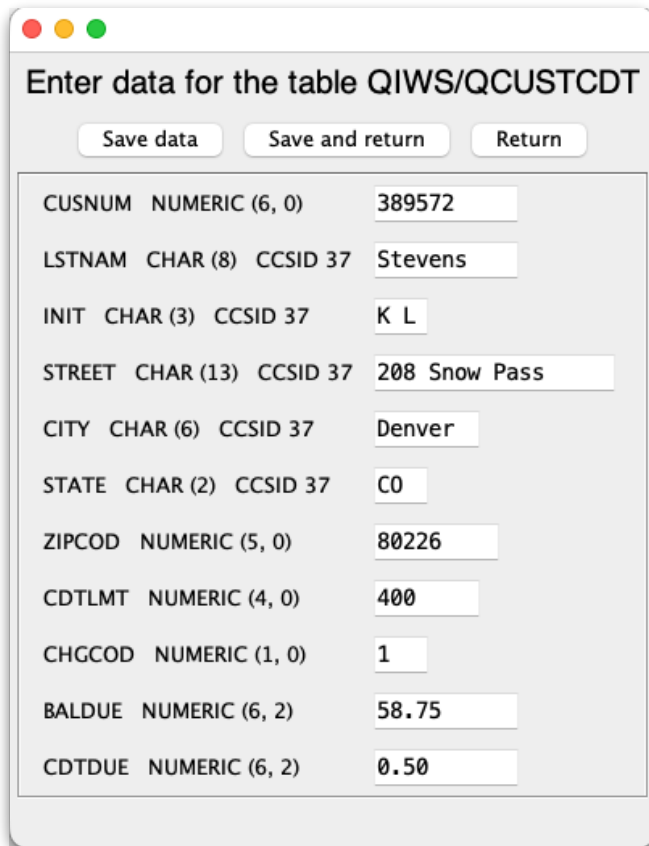
or

```
hex(BIN01) >= 1A
```

and the like.

Changing a record

To change a record select its row and press button *Edit selected*. Another window is displayed with a list of field names where input fields contain values from the row (record). The user can change contents of the input fields and press a button.



Field Name	Data Type	Value
CUSNUM	NUMERIC (6, 0)	389572
LSTNAM	CHAR (8) CCSID 37	Stevens
INIT	CHAR (3) CCSID 37	K L
STREET	CHAR (13) CCSID 37	208 Snow Pass
CITY	CHAR (6) CCSID 37	Denver
STATE	CHAR (2) CCSID 37	CO
ZIPCOD	NUMERIC (5, 0)	80226
CDTLMT	NUMERIC (4, 0)	400
CHGCOD	NUMERIC (1, 0)	1
BALDUE	NUMERIC (6, 2)	58.75
CDTDUE	NUMERIC (6, 2)	0.50

Button *Save data* writes the record to the file and the window remains displayed.

Button *Save and return* rewrites the record and returns the preceding window.

Button *Return* just returns the preceding window. The same is done at closing the window.

Input fields must have values according to the field type as defined for the record field.

- For character type (CHAR, VARCHAR, CLOB) empty values are acceptable. Values must correspond to the character set defined in the file for the field.
 - UTF-8 for CCSID 1208,
 - UTF-16 for CCSID 1200,
 - UCS-2 for CCSID 13488,
 - English for CCSID 37,
 - Czech for CCSID 870,
 - apod.
- Values of the types DEC, DECIMAL, NUMERIC may contain only digits, decimal point, and the sign before the number. Decimal comma is considered error.
- Values of the types INT, INTEGER, BIGINT may contain only digits and the sign before the number.
- Values of the type DATE must be entered in the format ISO, thus YYYY-MM-DD, e.g. 2025-02-15.
- Values of the type TIME must be entered in the format ISO, thus HH:MM:SS, e.g. 19:31:05.
- Values of the type TIMESTAMP must be entered in the format ISO, thus YYYY-MM-DD HH:MM:SS.MMMMMM, e.g. 2024-04-05 23:59:59.999999.
- Values of types BINARY or VARBINARY are entered in couples of hexadecimal characters. No check is done if they are correct. If the user enters an invalid character, 0 is assumed. The input fields are therefore twice as long.
- NULL value is entered how it is defined in application parameters, in our example it is a hyphen.

Creating the first record in empty file

Press button *Insert new row* in the window with empty list of records. The window with *empty* input fields is displayed. There enter values of corresponding types (see above).

Button *Save data* writes the new record to the file and the current window remains displayed.

Button *Save and return* writes the new record and a returns to the preceding window.

Button *Return* just returns to the preceding window. The same is done at closing the window.

Changing a cell (field in a record)

A field can be updated directly in the list of records using double click. The cell in the field is then enabled for editing (rewriting).

Table QIWS/QCUSTCDT											
Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).											
RRN	CUSNUM	LSTNAM	INIT	STREET	CITY	STATE	ZIPCOD	CDTLMT	CHGCOD	BALDUE	CDTDUE
4	938485	Johnson	J A	3 Alpine Way	Helen	GA	30545	9999	2	3987.50	0.50
11	192837	Lee	F L	5963 Oak St	Hector	NY	14841	700	2	489.50	0.50
3	392859	Vine	S S	P0 Box 79	Broton	VT	5046	700	1	439.00	0.00
8	475938	Doe	J W	59 Archer Rd	Sutter	CA	95685	700	2	250.00	1.00
2	839283	Jones	B D	21B NW 135 St	Clay	NY	13041	400	1	100.00	0.00
6	389572	Stevens	K L	208 Snow Pass	Denver	CO	80226	400	1	58.75	0.50

Rewrite the value 0.50 with a new value, say 44.50, and leave the cell by pressing the *Enter* key or *Tab* key or click on another cell; the new value is stored to the field of the record.

Table QIWS/QCUSTCDT											
Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).											
RRN	CUSNUM	LSTNAM	INIT	STREET	CITY	STATE	ZIPCOD	CDTLMT	CHGCOD	BALDUE	CDTDUE
4	938485	Johnson	J A	3 Alpine Way	Helen	GA	30545	9999	2	3987.50	0.50
11	192837	Lee	F L	5963 Oak St	Hector	NY	14841	700	2	489.50	0.50
3	392859	Vine	S S	P0 Box 79	Broton	VT	5046	700	1	439.00	0.00
8	475938	Doe	J W	59 Archer Rd	Sutter	CA	95685	700	2	250.00	1.00
2	839283	Jones	B D	21B NW 135 St	Clay	NY	13041	400	1	100.00	0.00
6	389572	Stevens	K L	208 Snow Pass	Denver	CO	80226	400	1	58.75	44.50

Note: Values of types BINARY or VARBINARY are entered in *couples of hexadecimal characters*. If the user enters an invalid character, 0 is stored instead. The input fields are therefore twice as long.

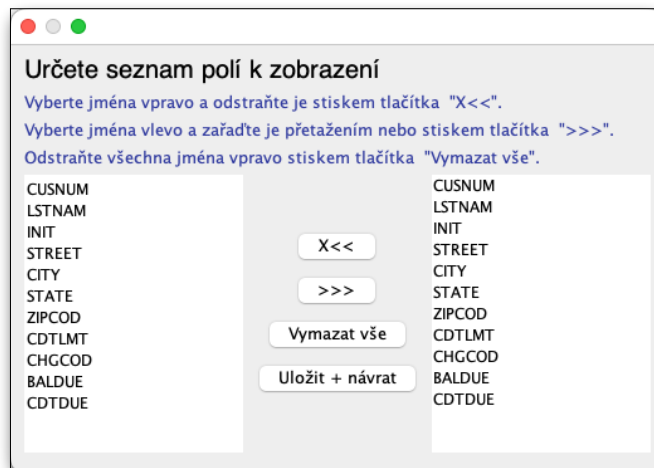
Deletion of a record

Select a row (record) from the list and press the button *Delete selected*. The record is removed from the list and deleted from the file.

Note: The deleted record is not accessible any more but occupies certain place in the data space. Its RRN number is also not available. The data space can be compressed and reordered by CL command RGZPFM (Reorganize Physical File Mbr) or by copying to another file and back by CL command CPYF.

Selection of columns

This function enables displaying a reduced set and different ordering of columns.



Button *Select columns* shows a window with two frames and buttons between.

The left frame contains always the complete list of fields (columns) of the database file. The right frame contains the same list at first. The user can write another list in the right frame, possibly shortened, or reordered.

- Button *X<<* removes selected names from the right frame.
- Drag selected names from the left frame into a suitable position in the right frame or copy them to the end of the right frame by pressing button *>>>*.
- Button ***Clear all*** clears the whole contents of the right frame.
- Button ***Save + return*** saves the list of field names from the right frame to a text file in the directory *columnfiles* and shows the table with the columns selected and ordered this way.

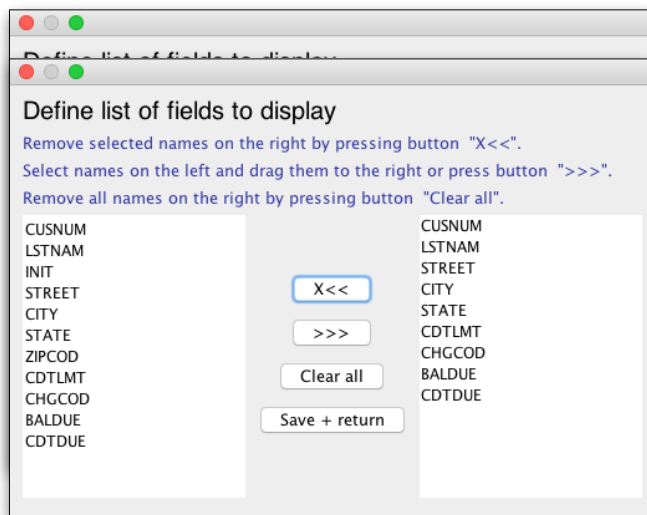
A single name from the list is selected by left mouse click. A contiguous group of names is selected by holding *Shift* key and left mouse clicking. Non-contiguous names are selected by holding *Ctrl* key (*Cmd* key in macOS) and left mouse clicking.

All functions *except for inserting a new row* can be performed in the reduced selection.

The list of selected fields amended by separating commas is stored in a work text file suffixed with **.col**. The file is placed in the directory *columnsfiles*. This list and the selection is valid until another selection is made on the file.

Example

After pressing the button *Select columns* the following window is displayed. We select field



Select fields ZIPCOD and INT in the right frame.

Press button X<<, the right field list is shortened.

Press button *Save + return* the resulting list is stored in the text file and the list of records is displayed with a new field configuration.

The fiels list is stored to the work file named `QIWS-QCUSTCDT.col` of the following form.

, CUSNUM, LSTNAM, INIT, CDTLMT, CHGCOD, BALDUE, CDTDUE

Note: The first comma separates the list from the field/column RRN, which is always displayed.

Next picture shows the result.

Table QIWS/QCUSTCDT

Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).

RRN	CUSNUM	LSTNAM	STREET	CITY	STATE	CDTLMT	CHGCOD	BALDUE	CTDUE
4	938485	Johnson	3 Alpine Way	Helen	GA	9999	2	3987.50	0.50
11	192837	Lee	5963 Oak St	Hector	NY	700	2	489.50	0.50
3	392859	Vine	P0 Box 79	Broton	VT	700	1	439.00	0.00
8	475938	Doe	59 Archer Rd	Sutter	CA	700	2	250.00	1.00
2	839283	Jones	21B NW 135 St	Clay	NY	400	1	100.00	0.00
6	389572	Stevens	208 Snow Pass	Denver	CO	400	1	58.75	44.50

Enter condition WHERE for row selection and press Refresh.

BALDUE >= 30 and CHGCOD < 3

Enter condition ORDER BY for row ordering and press Refresh.

BALDUE DESC

```

SELECT RRN(QCUSTCDT) as RRN, CUSNUM, LSTNAM, STREET, CITY, STATE, CDTLMT, CHGCOD, BALDUE, CTDUE
FROM QIWS/QCUSTCDT
WHERE BALDUE >= 30 and CHGCOD < 3
ORDER BY BALDUE DESC
FETCH FIRST 200 ROWS ONLY

```

Exit

Insert new row

Edit selected

Refresh

Delete selected

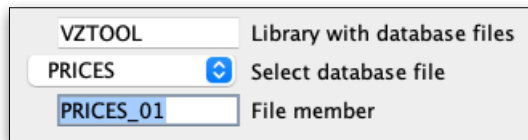
Select columns

Data members of physical and logical files

The application can process data members of physical and logical files as if there were files. An alias object must be created in the library for the member whose name differs from the default of the file (*FIRST or file name). The *application* creates the alias object *itself* if it does not exist.

Note: Deleting records from these members is not possible.

For example, the user selects the file VZTOOL/PRICES in application parameters and changes the input field *File member* to PRICES_01.



The screenshot shows a window with three input fields. The first field is labeled 'VZTOOL' and has a dropdown arrow. The second field is labeled 'PRICES' and has a dropdown arrow. The third field is labeled 'PRICES_01' and has a dropdown arrow. To the right of these fields are labels: 'Library with database files', 'Select database file', and 'File member'.

If the member exists, the application automatically creates a new alias object named identically PRICES_01 and displays the list of records contained in the member (or empty member). As soon as the application leaves the window with the displayed data, it deletes the corresponding alias object.

Alias objects

The alias objects are physical files (object type *FILE) with attribute DDMF. DDMF (Distributed Data Management File) is an object serving in access to a Remote Location from a Local Location. It is an object from SNA, APPC architecture. Here, both locations are the same and the alias object serves as a medium of access to the data file.

The user can also create alias objects using SQL statements using SQL statement CREATE ALIAS. A different means than this application must be used, e.g. STRSQL CL command. For example, if the physical file PRICES has members PRICES_01, PRICES_02, the user creates alias objects using the following SQL statements.

```
DROP ALIAS PRICES_01
CREATE ALIAS PRICES_01 FOR VZTOOL.PRICES(PRICES_01)
DROP ALIAS PRICES_02
CREATE ALIAS PRICES_02 FOR VZTOOL.PRICES(PRICES_02)
```

The alias names need not be the same as the member names but it is practical.

The objects in the library look like this:

PRICES	*FILE	PF-DTA
PRICES_01	*FILE	DDMF
PRICES_02	*FILE	DDMF

The same method can be used for logical file members. If the logical file is named PRICESL and has members PRICESL_01, PRICESL_02, the alias objects look like this:

PRICESL	*FILE	PF-DTA
PRICESL_01	*FILE	DDMF
PRICESL_02	*FILE	DDMF

Data types CLOB and BLOB

These data types large objects - LOB containing data of so called advanced types CLOB (Character Large Object) and BLOB (Binary Large Object).

Columns of these types are not displayed in the list of rows in the table because the amount of data is too large for a cell in the table row. Data is displayed only at updating in a separate window when the column already contains some value that can be displayed. This means a text in case of CLOB or a bit map of certain type (e.g. a photo) in case of BLOB.

Columns can obtain a value either at insertion of a new record or at updating of an existing record.

In the following explanation we will work with a table VZTOOL/BLOBCLOB created by the following statement.

```
CREATE TABLE VZTOOL.BLOBCLOB (  
    COL0 DECIMAL (7, 0),  
    CLOB1 CLOB (1000),  
    COL2 CHAR (5),  
    CLOB2 CLOB (500),  
    BLOB1 BLOB (10000000),  
    BLOB2 BLOB  
)
```

Note the sizes in parentheses at the columns of CLOB and BLOB types.

Columns CLOB1 and CLOB2 are relatively short with capacities 1000 and 500 characters respectively. We can observe attempts of filling them with smaller or larger files and behavior of the application when the capacity is exceeded.

Columns BLOB1 and BLOB2 are long enough with capacities 10 million bytes and no explicit size (1 MB is the default value) respectively for very large files.

Buttons have their default character set – CCSID – because it was not specified at the column definitions:

File name	Column type	CCSID	Column name	Length	Dec. pos.
BLOBCLOB	DECIMAL	–	COL0	7	0
BLOBCLOB	CLOB	870	CLOB1	1000	–
BLOBCLOB	CHAR	870	COL2	5	–
BLOBCLOB	CLOB	870	CLOB2	500	–
BLOBCLOB	BLOB	65535	BLOB1	10000000	–
BLOBCLOB	BLOB	65535	BLOB2	1048576	–

Insert a new row

Only columns of normal types are displayed in the rows. The file is empty at first.

Table VZTOOL/BLOBCLOB
Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).

RRN	COL0	COL2
-----	------	------

Exit

Insert new row

Edit selected

Refresh

Delete selected

Select columns

After pressing the button *Insert new row* a window with the list of columns is displayed.

Enter data for the table VZTOOL/BLOBCLOB

Save data

Save and return

Return

COL0 DECIMAL (7, 0)

COL2 CHAR (5) CCSID 870

COLB1 CLOB (1000) CCSID 870

COLB2 CLOB (500) CCSID 870

BLOB1 BLOB (10000000)

BLOB2 BLOB (1048576)

Normal columns are placed at the beginning even if they were not defined in this sequence. Underneath follow columns of CLOB type and then BLOB types. Numbers in parentheses denote capacity - maximum size of the contents. For CLOB number of characters, for BLOB inumber of bytes. Columns of character types specify their character set (CCSID).

Enter data in columns COL1 and COL2 (e.g. number 1 and character A) and press button *Save data* or *Save and return*. Data is stored to the record whereas the large columns remain empty (have now NULL value) and the list of rows is displayed with the entered "normal" data.

Table VZTOOL/BLOBCLOB
Change a cell value: Double click, rewrite the cell value and press ENTER (or click TAB or another cell).

RRN	COL0	COL2
1	1	A

Updating a row

Select the row in the table and press button *Edit selected*. The window for updating the columns is displayed.

Enter data for the table VZTOOL/BLOBCLOB

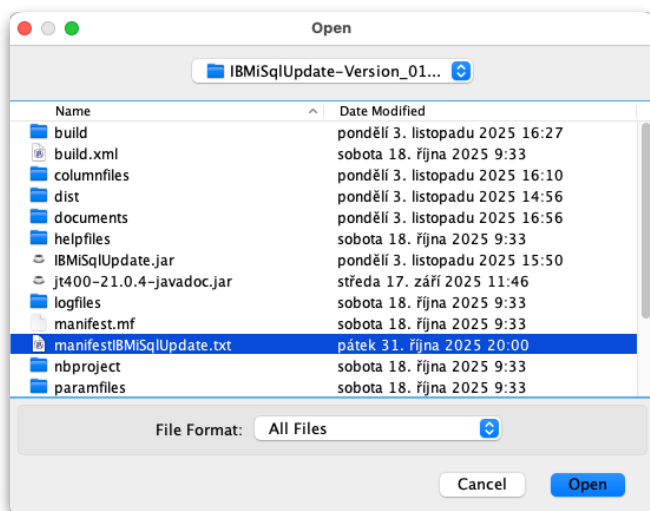
Save data Save and return Return

COL0	DECIMAL (7, 0)	1
COL2	CHAR (5) CCSID 870	A
CLOB1	CLOB (1000) CCSID 870	CLOB1
CLOB2	CLOB (500) CCSID 870	CLOB2
BLOB1	BLOB (10000000)	BLOB1
BLOB2	BLOB (1048576)	BLOB2

Normal columns can be changed as usual. Large object columns can be changed using *buttons* with their names (instead of input fields).

Filling column of CLOB type

Contents of the column is NULL (the first record is inserted). Press button CLOB2 and the window for PC file selection is displayed where the desired file can be successively found in directories.



If we select an input file, its contents is stored in the column and a message with the size of the file is displayed. Text of the column is not shown for now.

Enter data for the table VZTOOL/BLOBCLOB

Save data Save and return Return

COL0 DECIMAL (7, 0) 1

COL2 CHAR (5) CCSID 870 A

CLOB1 CLOB (1000) CCSID 870 CLOB1

CLOB2 CLOB (500) CCSID 870 CLOB2

BLOB1 BLOB (10000000) BLOB1

BLOB2 BLOB (1048576) BLOB2

Content of the file or part of it was loaded into the column. Length is 113

To see the column contents *press the button CLOB2 again*. Then the window with the text of the column is displayed.

Column CLOB2 - UTF-8

Start of text: 1 Length of text: 113 Find text:

Manifest-Version: 1.0
Class-Path: ./jt400-21.0.4.jar ./jt400-21.0.4-javadoc.jar
Main-Class: update/U_MainWindow

Return Refresh Save data Get File Put File Page Setting Print

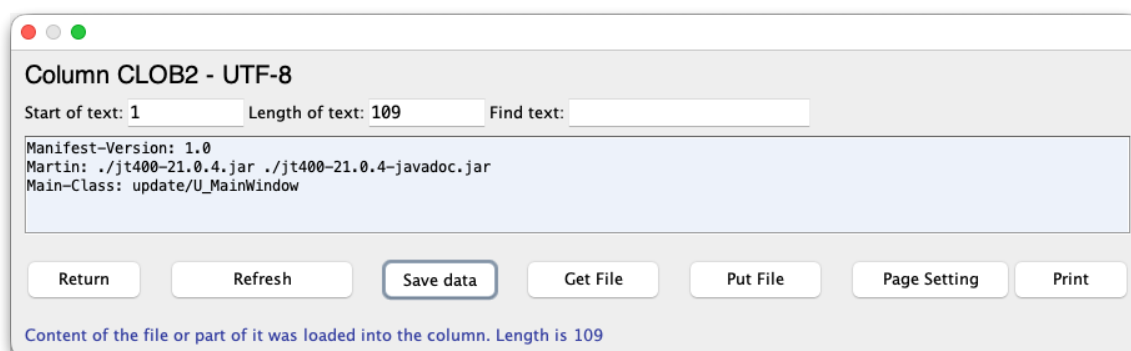
Column length is 113.

This example shows the desired case when the input file is shorter (113) than the capacity of the column (500). Data in input fields are set to initial values:

- *Start of text* is 1.
- *Length of text* is 113 (including character *newline* at the end), i.e. length of all text contained in the column CLOB2.
- *Find text* is empty.

Overwriting text

The text in the window can be rewritten or added and saved with *Save data*. For example, change `Class-Path` to `Martin` and press *Save data*.

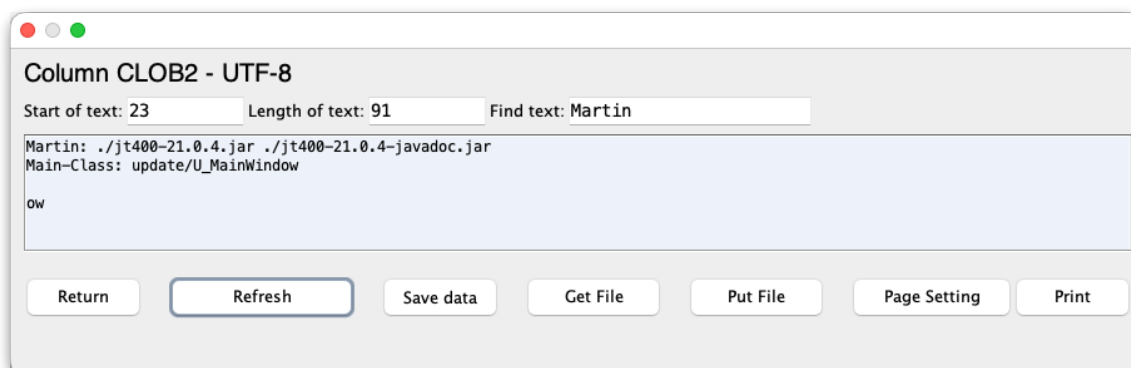


Text length changed to 109 because text `Martin` is 4 characters shorter than the rewritten text `Class-path`. The overall length of the column remains unchanged. The whole text of the column was overwritten only by the displayed section of length 109 (including the *newline* character).

Finding text by a pattern

If we enter *Martin* (pattern to be found) to *Find text*, and press *Refresh* or *Enter*, the text is searched for the pattern from the position 1 in *Start of text*.

If the pattern is found, the text from the beginning of found data is displayed. Position 23 of the found data is written to *Start of field*. Length 91 is written to *Length of text*. It is the length of *displayed* text (not the whole text).

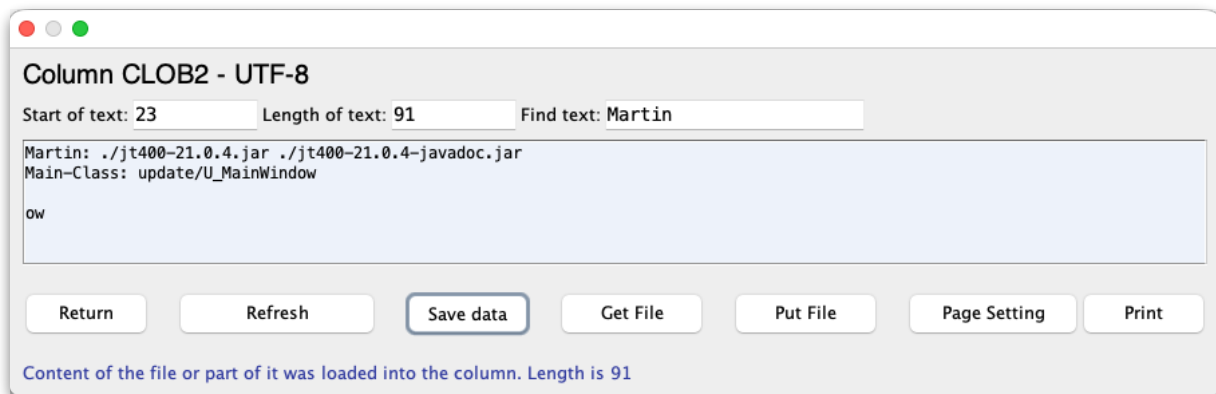


Text `Martin` is 4 characters shorter than the replaced text `Class-path`. Therefore the resulting text was shifted to the left by 4. Only the text from the position 23 is displayed but the overall length 113 remains unchanged ($22 + 91 = 113$). The text *ow* is end part of the remainder from the original text (first two characters of the remainder are *newlines*).

Note: Positions are counted from 1.

Saving the changed text

Pressing button *Save data* overwrites the column contents by the text just displayed in the window from the *Start position* in the *Length of text*.



The screenshot shows a window titled "Column CLOB2 - UTF-8". At the top, there are three input fields: "Start of text:" with the value "23", "Length of text:" with the value "91", and "Find text:" with the value "Martin". Below these fields is a text area containing the following text:

```
Martin: ./jt400-21.0.4.jar ./jt400-21.0.4-javadoc.jar
Main-Class: update/U_MainWindow

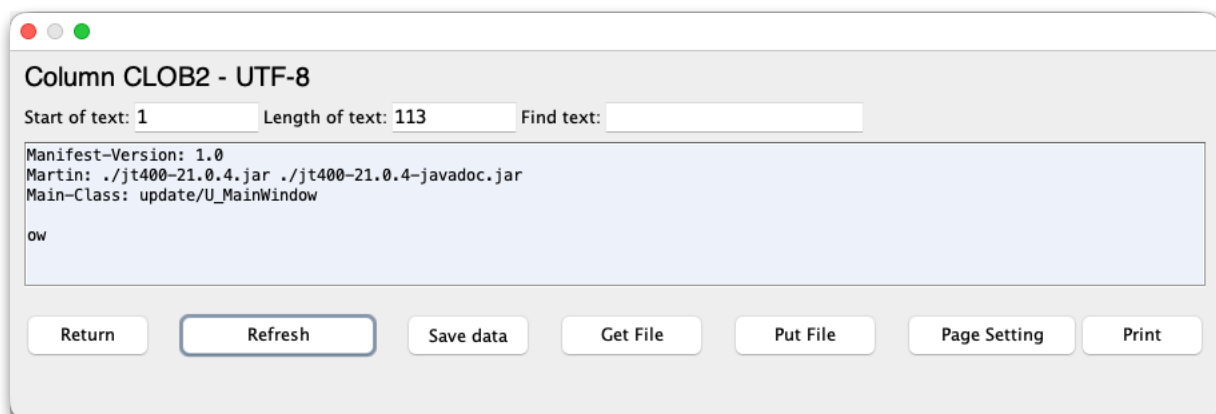
Ow
```

At the bottom of the window, there is a row of buttons: "Return", "Refresh", "Save data", "Get File", "Put File", "Page Setting", and "Print". The "Save data" button is highlighted with a blue border. Below the buttons, there is a status bar that reads: "Content of the file or part of it was loaded into the column. Length is 91".

Pressing button *Return* also overwrites column contents like the button *Save data*. Moreover the window with the list of columns is then displayed.

If we display the column by the button CLOB2 again, the same picture is displayed but with empty field *Find text*.

If we change the position to 1 and the length to a greater value (e.g. 200) and press *Refresh* button, text in length of 113 characters is displayed.



The screenshot shows the same window titled "Column CLOB2 - UTF-8". The input fields now show "Start of text:" with the value "1", "Length of text:" with the value "113", and "Find text:" which is empty. The text area contains the following text:

```
Manifest-Version: 1.0
Martin: ./jt400-21.0.4.jar ./jt400-21.0.4-javadoc.jar
Main-Class: update/U_MainWindow

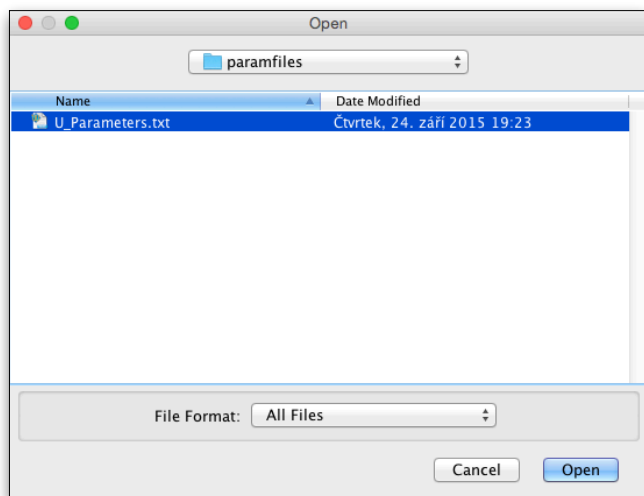
Ow
```

The buttons at the bottom are the same, but now the "Refresh" button is highlighted with a blue border. The status bar is not visible in this screenshot.

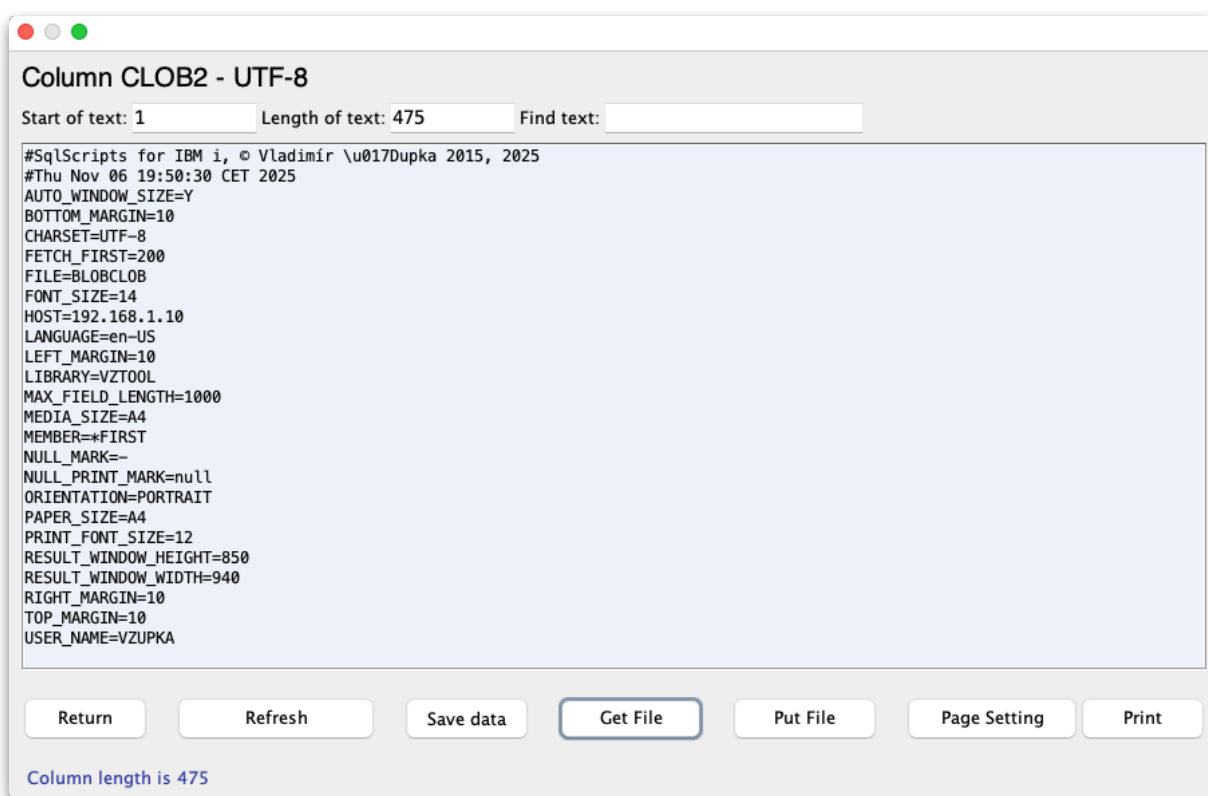
Changing column contents

The button *Get file* allows to overwrite the whole contents of the column by contents of a text file.

For example, we want to see the new text from position 1. So enter 1 to *Start of text* and press button *Get file*. A window with the list of files is displayed from which we select a file.

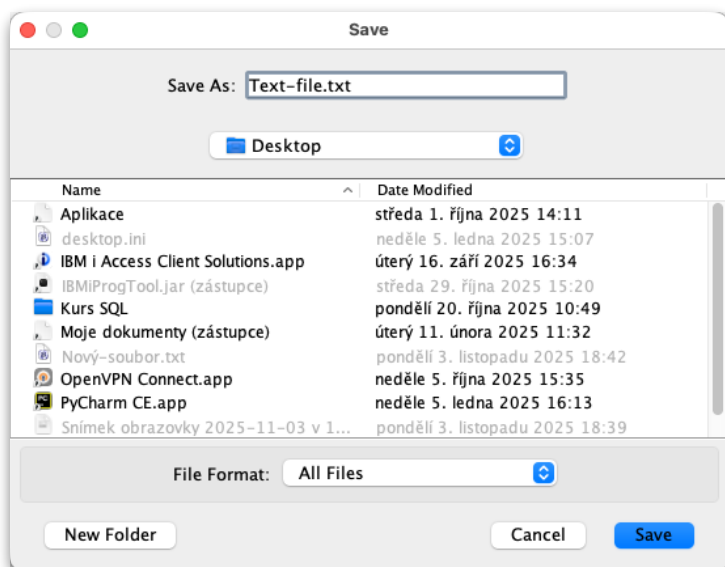


After pressing *Open* the text taken from the selected file is displayed in the window.



Saving column value to a file

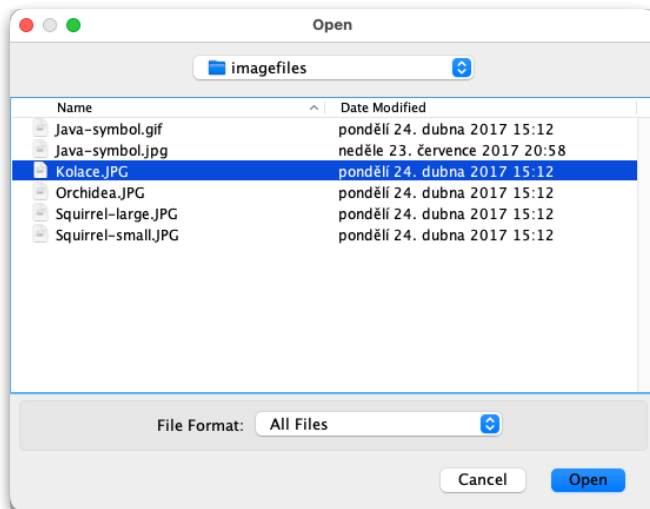
The button *Put file* invokes a window where we enter the name of a file to the input field *Save As:* and select a directory. Then after pressing *Save* button the file is created in the directory (Desktop).



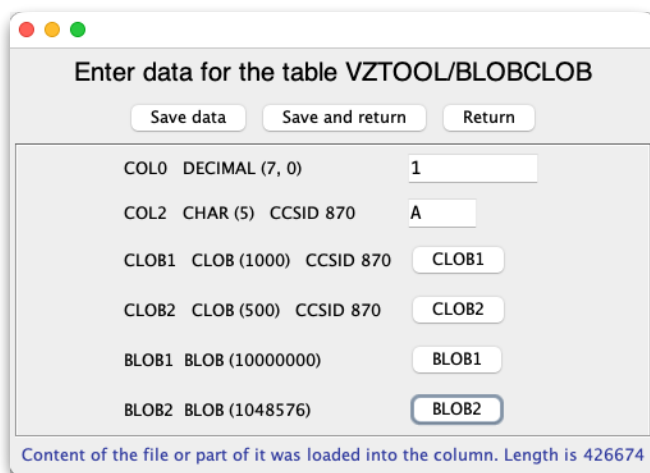
The new file will be saved in the *character set* defined in application parameters.

Filling column of BLOB type

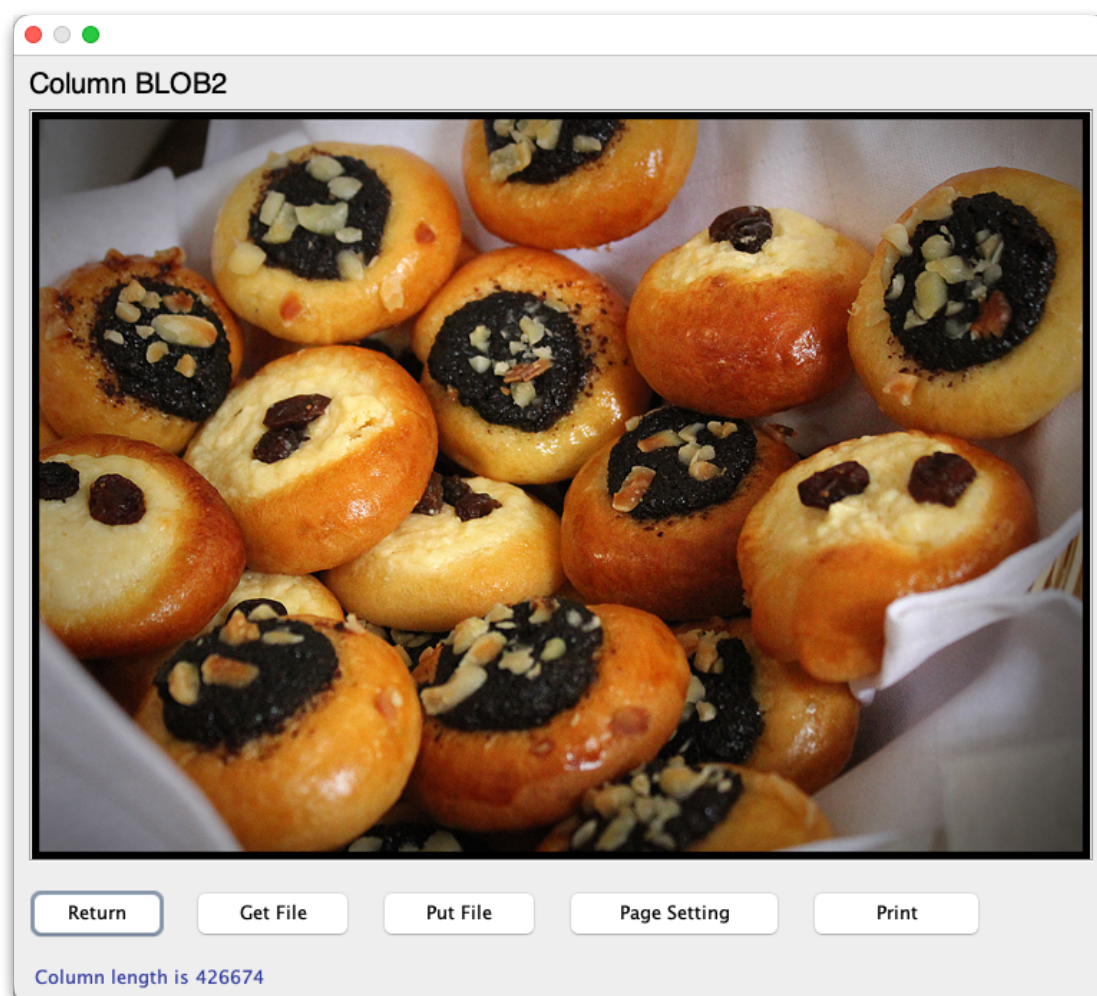
Press button BLOB2 in the list of columns. If the column is empty, a window is displayed from which we select a file, this time type JPG (photo).



New contents of the column is not shown, but only a message about loading the contents and length of data.



Press button BLOB2 again and new contents of the column is displayed.

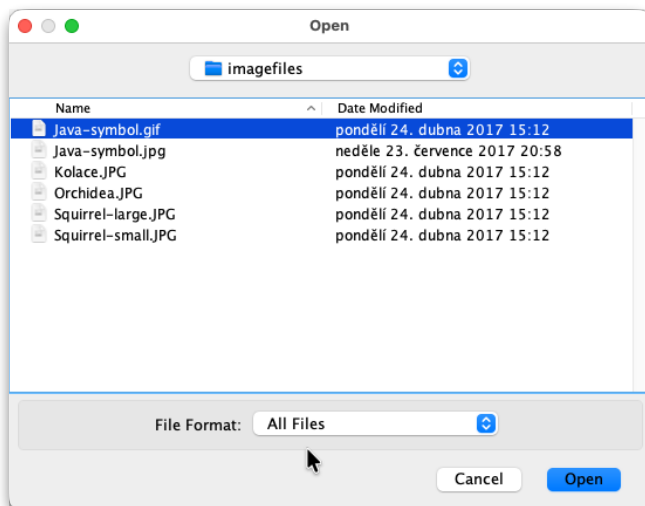


If the column had already a non-null contents, then the image is displayed immediately.

Note: Bitmap files of types JPEG, JPG, GIF, PNG, BMP, WBMP can be displayed as a picture, other types cannot.

Changing contents of a column

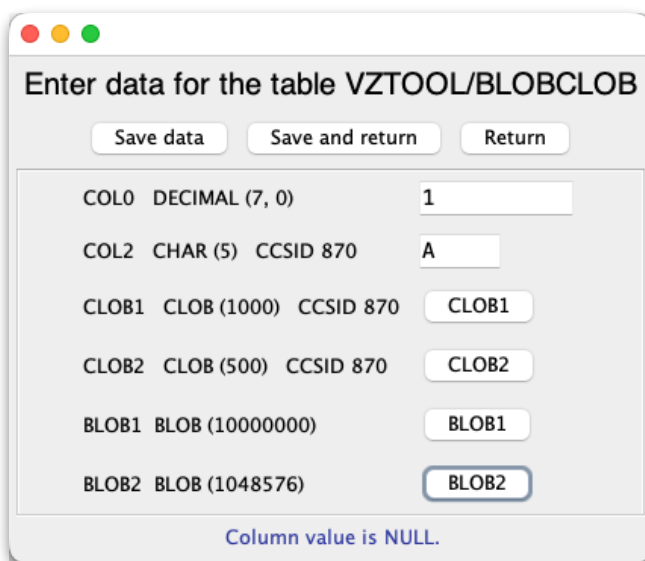
Press button *Get file* and select a file from the list, this time type GIF.



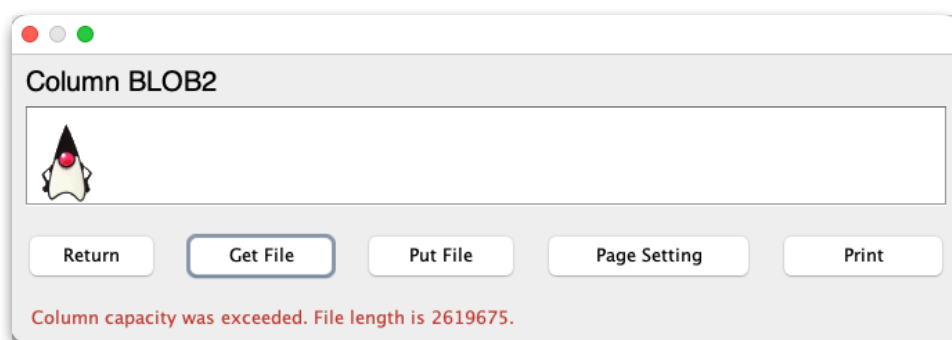
After pressing *Open*, the contents is loaded from the selected file and displayed with a message reporting the length.



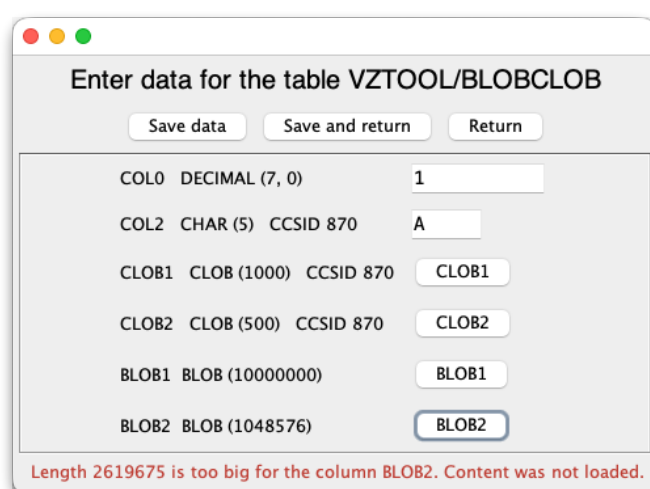
Note: If we press the *Cancel* button, the column gets NULL value.



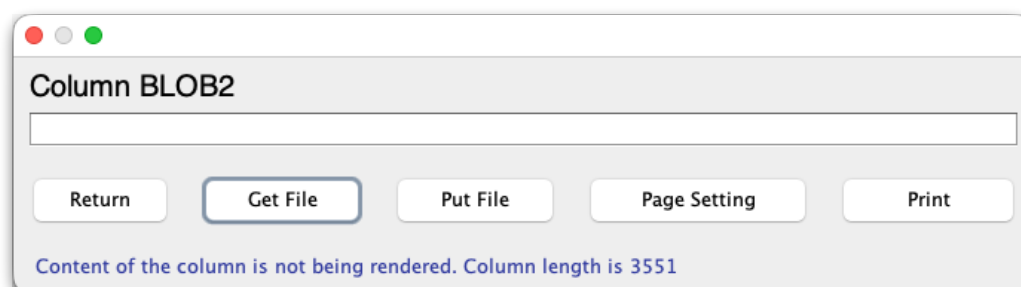
If the contents of the selected file is greater than the capacity of the column, a message tells that the column is not capable to absorb contents of the new file. The actual picture remains visible.



Button *Return* returns the window with column list where the message tells that the column capacity was exceeded and that the file contents was not loaded.



If the contents of the column is not displayed as a picture, the window is shown with a message telling the length of the column.



Problems with big columns

Transfer of large volumes of data between the database and the personal computer lasts rather long, especially if flowing through Internet. Be patient until the contents is transferred or until it is certain that it does not fit.

Sometimes it may happen that the connection between the personal computer and the database is interrupted at the moment when the record is locked for update. In such a case the application is waiting for a response from the database. If no response comes a message is reported in the file *err.txt* in directory *logfiles* or in the window.

This situation can be remedied if we cancel the job in which the communication was run. It is found using the CL command WRKOBJLCK (Work with Object Locks):

```
WRKOBJLCK OBJ(VZTOOL/BLOBCLOB) OBJTYPE(*FILE) MBR(*FIRST)
```

Work with Member Locks

File : BLOBCLOB

Type : PHY

Library . . . : VZTOOL

ASP device . . : *SYSBAS

System: XASSIST

Type options, press Enter.

4=End job 5=Work with job 8=Work with job locks

Opt	Member	Job	User	Lock Type	Lock	Status	Share
	BLOBCLOB	QZDASOINIT	QUSER	MBR	*SHRRD	HELD	
				MBR	*SHRRD	HELD	
				MBR	*SHRRD	HELD	
				MBR	*SHRRD	HELD	
				DATA	*SHRRD	HELD	
				DATA	*SHRUPD	HELD	
				DATA	*SHRUPD	HELD	
				DATA	*SHRUPD	HELD	

F3=Exit F5=Refresh F11=Display thread data F12=Cancel

Bottom

The job is named QZDASOINIT and can be found using CL command WRKACTJOB. There may be multiple jobs of that name depending on how many users are working with the database. There may be more such jobs, according to how many users work with the file. Find the job of the user who works with the file (BLOBCLOB in the example). Ending the job releases the record and the application can be launched again and work with the same record.

Type options, press Enter.						
2=Change		3=Hold	4=End	5=Work with	6=Release	7=Display message
8=Work with spooled files		13=Disconnect ...				
Current						
Opt	Subsystem/Job	User	Type	CPU %	Function	Status
	QUSRWRK	QSYS	SBS	.0		DEQW
	QPOZSPWP	QSECOFR	BCI	.0	PGM-sshd	SELW
	QSLICEV	QSYS	BCH	.0		DEQW
	QSCWCHMS	QUSER	BCH	.0		DEQW
	QZDASOINIT	QSECOFR	PJ	.0		TIMW
	QZDASOINIT	VZUPKA	PJ	.0		TIMW
	QZRCSRVS	QSECOFR	PJ	.0		TIMW
	QZRCSRVS	VZUPKA	PJ	.0		TIMW